DISEÑO DE SERVICIOS WEB PARA EDUSMART

14.1. Introducción

Para garantizar el funcionamiento de las principales funcionalidades de EduSmart, se propone diseñar un conjunto de servicios web que permitan la comunicación entre la interfaz de usuario, la lógica de negocio y los componentes encargados de gestionar la información académica.

EduSmart es una plataforma orientada a facilitar la consulta de recursos educativos, la identificación de necesidades académicas y la generación de recomendaciones relacionadas con los intereses de aprendizaje de los estudiantes. Por esta razón, los servicios web deben permitir el intercambio organizado de información entre los diferentes componentes del sistema.

El diseño se plantea principalmente bajo el enfoque **REST (Representational State Transfer)**, debido a que permite construir servicios mediante recursos identificables por URL, utilizar métodos HTTP estandarizados y transmitir información mediante formatos como JSON.

A través de estos servicios se busca atender las necesidades de los tres perfiles principales de EduSmart: estudiantes, asesores académicos y administradores.

14.2. Objetivo del diseño de servicios web

Definir los servicios necesarios para soportar las funcionalidades principales de EduSmart, estableciendo las operaciones que permitirán consultar, registrar, actualizar y relacionar la información académica de la plataforma.

De esta manera, se busca establecer una comunicación organizada entre las interfaces, los servicios de aplicación, la información almacenada y los componentes semánticos basados en la ontología OWL y el grafo de conocimiento de Neo4j.

14.3. Arquitectura propuesta para los servicios web

Se propone una arquitectura en la que los usuarios interactúan con la plataforma mediante una interfaz web. Esta interfaz realiza solicitudes a una API REST, encargada de recibir las peticiones, validar los datos y ejecutar las operaciones correspondientes.

La API se comunica con los componentes responsables de administrar los recursos educativos, las necesidades académicas, las recomendaciones y las relaciones semánticas.

Los componentes principales son:

1. **Interfaz web:** permite que los estudiantes, asesores y administradores utilicen las funcionalidades de EduSmart.
2. **API REST:** recibe las solicitudes HTTP y organiza la comunicación entre la interfaz y los servicios.
3. **Servicios de aplicación:** ejecutan las operaciones relacionadas con recursos, necesidades, recomendaciones y administración.
4. **Base de datos:** almacena la información de usuarios, recursos educativos, categorías y necesidades académicas.
5. **Ontología OWL:** representa los conceptos, propiedades y reglas del dominio académico.
6. **Grafo de conocimiento Neo4j:** permite consultar las relaciones entre necesidades, temas, recursos y recomendaciones.

14.4. Convenciones generales de los servicios REST

Para mantener una estructura uniforme, se propone utilizar la siguiente dirección base:

`https://api.edusmart.com/api/v1`

Esta dirección es ilustrativa y representa la URL que podría utilizarse cuando los servicios sean implementados y publicados.

Los servicios utilizarán principalmente los siguientes métodos HTTP:

MÉTODO “HTTP”	PROPÓSITO	APLICACIÓN EN EDUSMART
GET	es en consultar información	en buscar recursos, categorias, necesidad
POST	en registrar información o en ejecutar una operación	agregar recursos o registrar una necesidad
PUT	en reemplazar completamente un recurso existente	en actualizar todos los datos de un recursos educativo
PATCH	en modificar parcialmente un recursos e	en cambiar el estado o la descripción de un recursos
DELETE	en eliminar un recurso	en retirar un recursos del catálogo cuando corresponda

Se propone utilizar JSON como formato principal para el intercambio de información, debido a que es ligero, legible y compatible con aplicaciones web desarrolladas con JavaScript.

14.5. Identificación de los servicios necesarios

Para soportar las funcionalidades principales de EduSmart se identificaron diez servicios web. Cada uno responde a una necesidad específica de la plataforma.

Servicio 1. Consulta de recursos educativos

Nombre: Servicio de consulta de recursos.

Propósito: permitir que los estudiantes y asesores consulten los materiales educativos disponibles en EduSmart.

Recurso o funcionalidad atendida: catálogo de recursos educativos, búsqueda y consulta de materiales.

Endpoint propuesto:

```
GET /api/v1/recursos
```

Datos de entrada:

- Palabra clave para realizar la búsqueda.
- Identificador de categoría.
- Tema académico.
- Número de página.
- Cantidad de resultados por página.

Ejemplo de solicitud:

```
GET /api/v1/recursos?categoria=lectura&buscar=comprension
```

Datos de salida:

- Identificador del recurso.
- Nombre del material.
- Descripción.
- Categoría.
- Tema relacionado.
- Nivel de dificultad.
- Estado de disponibilidad.

Método HTTP esperado: GET.

Usuario o componente que lo utiliza: estudiantes, asesores académicos e interfaz de recursos educativos.

Servicio 2. Consulta del detalle de un recurso

Nombre: Servicio de detalle de recursos.

Propósito: proporcionar información completa sobre un material educativo seleccionado por el usuario.

Recurso o funcionalidad atendida: consulta individual de recursos educativos.

Endpoint propuesto:

```
GET /api/v1/recursos/{id}
```

Datos de entrada:

- Identificador único del recurso.

Ejemplo:

```
GET /api/v1/recursos/R001
```

Datos de salida:

- Identificador.
- Nombre del recurso.
- Descripción completa.
- Categoría.
- Tema.
- Nivel de dificultad.
- Necesidades académicas relacionadas.
- Enlace al material, si está disponible.

Método HTTP esperado: GET.

Usuario o componente que lo utiliza: estudiantes, asesores académicos y módulo de detalle del recurso.

Servicio 3. Consulta de categorías educativas

Nombre: Servicio de categorías.

Propósito: proporcionar las categorías disponibles para clasificar y organizar los recursos educativos.

Recurso o funcionalidad atendida: filtros y clasificación del catálogo.

Endpoint propuesto:

`GET /api/v1/categorias`

Datos de entrada:

- Identificador de categoría, cuando se requiera una consulta específica.
- Estado de la categoría, si se implementa dicho filtro.

Datos de salida:

- Identificador de categoría.
- Nombre.
- Descripción.
- Estado.

Método HTTP esperado: GET.

Usuario o componente que lo utiliza: estudiantes, asesores, administradores e interfaz del catálogo.

Servicio 4. Consulta de necesidades académicas

Nombre: Servicio de necesidades académicas.

Propósito: permitir que los estudiantes consulten las necesidades de aprendizaje contempladas en la plataforma y seleccionen aquella que desean fortalecer.

Recurso o funcionalidad atendida: módulo de necesidades académicas.

Endpoint propuesto:

`GET /api/v1/necesidades`

Para registrar la selección de una necesidad se propone una operación adicional:

`POST /api/v1/estudiantes/{id}/necesidades`

Datos de entrada para consultar:

- Identificador de necesidad, si se requiere una consulta individual.
- Palabra clave, cuando se implemente búsqueda.

Datos de entrada para registrar la selección:

- Identificador del estudiante.
- Identificador de la necesidad académica seleccionada.

Datos de salida:

- Identificador de la necesidad.
- Nombre.

- Descripción.
- Confirmación de la selección, cuando se registre.

Métodos HTTP esperados: GET y POST.

Usuario o componente que lo utiliza: estudiantes, interfaz de necesidades académicas y servicio de recomendaciones.

Servicio 5. Generación de recomendaciones académicas

Nombre: Servicio de recomendaciones.

Propósito: identificar y presentar recursos educativos que puedan responder a las necesidades académicas seleccionadas por los estudiantes.

Recurso o funcionalidad atendida: módulo de recomendaciones personalizadas.

Endpoint propuesto:

```
GET /api/v1/recomendaciones?necesidadId=N001
```

Datos de entrada:

- Identificador de la necesidad académica.
- Identificador del estudiante, cuando la recomendación tenga en cuenta su contexto.
- Tema de interés, si corresponde.

Datos de salida:

- Identificador de la necesidad consultada.
- Nombre de la necesidad.
- Lista de recursos recomendados.
- Descripción de cada recurso.
- Tema relacionado.
- Justificación de la relación.
- Nivel de correspondencia, cuando se implemente un criterio de clasificación.

Método HTTP esperado: GET.

Usuario o componente que lo utiliza: estudiantes, asesores académicos, interfaz de recomendaciones y motor de consulta semántica.

Relación con la ontología y Neo4j: este servicio podría consultar las relaciones entre necesidades académicas, temas y recursos educativos representadas en la ontología OWL y el grafo de conocimiento. De esta manera, las recomendaciones no dependerían únicamente de asociaciones programadas manualmente, sino también de las relaciones definidas en el dominio.

Servicio 6. Consulta de relaciones semánticas

Nombre: Servicio de relaciones del conocimiento.

Propósito: consultar las conexiones existentes entre necesidades académicas, temas, recursos educativos y otros conceptos del dominio.

Recurso o funcionalidad atendida: sección de relaciones y visualización del conocimiento.

Endpoint propuesto:

```
GET /api/v1/relaciones?necesidadId=N001
```

Datos de entrada:

- Identificador de la necesidad académica.
- Tipo de relación, cuando se requiera filtrar.
- Identificador del recurso o tema, si se realiza una consulta específica.

Datos de salida:

- Identificador del concepto de origen.
- Nombre del concepto.
- Tipo de relación.
- Identificador del concepto relacionado.
- Nombre del concepto relacionado.

Método HTTP esperado: GET.

Usuario o componente que lo utiliza: estudiantes, asesores académicos, interfaz de relaciones y servicio de consulta del grafo Neo4j.

Servicio 7. Registro de recursos educativos

Nombre: Servicio de creación de recursos.

Propósito: permitir que el administrador incorpore nuevos materiales al catálogo de EduSmart.

Recurso o funcionalidad atendida: formulario de administración de recursos.

Endpoint propuesto:

```
POST /api/v1/recursos
```

Datos de entrada:

- Nombre del recurso.
- Descripción.
- Categoría.
- Tema relacionado.

- Nivel de dificultad.
- Enlace al material, cuando exista.

Datos de salida:

- Identificador asignado al nuevo recurso.
- Nombre registrado.
- Estado de la operación.
- Mensaje de confirmación.

Método HTTP esperado: POST.

Usuario o componente que lo utiliza: administrador e interfaz de administración.

Servicio 8. Actualización de recursos educativos

Nombre: Servicio de actualización de recursos.

Propósito: permitir que el administrador modifique la información de un material educativo cuando sea necesario.

Recurso o funcionalidad atendida: mantenimiento y actualización del catálogo.

Endpoint propuesto:

```
PATCH /api/v1/recursos/{id}
```

Datos de entrada:

- Identificador del recurso.
- Campos que se desean modificar, como nombre, descripción, categoría o estado.

Datos de salida:

- Identificador del recurso.
- Información actualizada.
- Estado de la operación.
- Mensaje de confirmación.

Método HTTP esperado: PATCH.

Usuario o componente que lo utiliza: administrador y servicio de gestión del catálogo.

Servicio 9. Eliminación o desactivación de recursos

Nombre: Servicio de retiro de recursos.

Propósito: permitir que el administrador retire del catálogo materiales que ya no se encuentren disponibles o que deban dejar de mostrarse.

Recurso o funcionalidad atendida: administración y mantenimiento de recursos educativos.

Endpoint propuesto:

```
DELETE /api/v1/recursos/{id}
```

Datos de entrada:

- Identificador del recurso que se desea retirar.
- Identificación del usuario administrador que ejecuta la operación, obtenida mediante el mecanismo de autenticación.

Datos de salida:

- Identificador del recurso.
- Estado de la operación.
- Mensaje de confirmación.

Método HTTP esperado: DELETE.

Usuario o componente que lo utiliza: administrador y servicio de gestión del catálogo.

Servicio 10. Consulta de servicios de apoyo académico

Nombre: Servicio de consulta de servicios académicos.

Propósito: permitir que los usuarios consulten los servicios institucionales que pueden complementar el acompañamiento académico de los estudiantes.

Recurso o funcionalidad atendida: consulta de servicios académicos y orientación complementaria.

Endpoint propuesto:

```
GET /api/v1/servicios-academicos
```

Datos de entrada:

- Tipo de servicio.
- Tema relacionado.
- Palabra clave, si se implementa búsqueda.

Datos de salida:

- Identificador del servicio.
- Nombre.
- Descripción.
- Tema relacionado.
- Información de contacto o enlace institucional, cuando esté disponible.

Método HTTP esperado: GET.

Usuario o componente que lo utiliza: estudiantes, asesores académicos e interfaz de servicios de apoyo.

14.6. Resumen general de los servicios REST

La siguiente tabla consolida los servicios propuestos y permite identificar de manera rápida las operaciones que soportan las funcionalidades de EduSmart.

N.	SERVICIO	ENDPOINT PRINCIPAL	MÉTODO “HTTP”	USUARIO O COMPONENTES
1	consulta de recursos	/api/V1/recursos	GET	estudiante y asesor
2	detalle de recurso	/api/V1/recursos/{ID}	GET	estudiante y asesor
3	consulta de categorías	/api/V1/categorías	GET	estudiante, asesor y administrador
4	necesidades académicas	/api/V1/necesidades	GET/POST	estudiante
5	recomendaciones	/api/V1/recomendaciones	GET	estudiante y asesor
6	relaciones semánticas	/api/V1/relaciones	GET	estudiante, asesor y neo4j
7	registro de recursos	/api/V1/recursos	POST	administrador
8	actualización de recursos	/api/V1/recursos/{ID}	PATCH	administrador
9	retiro de recursos	/api/V1/recursos/{ID}	DELETE	administrador
10	servicios académicos	/api/V1/servicios-académicos	GET	estudiante y asesor

14.7. Manejo de respuestas y errores de los servicios

Para facilitar la comunicación entre los componentes de EduSmart, se propone que los servicios utilicen códigos de estado HTTP que permitan identificar el resultado de cada solicitud.

CÓDIGO “HTTP”	SIGNIFICADO	SITUACIÓN DE USO
200 OK	solicitud procesada correctamente	consulta o actualización exitosa
201 created	recurso creado correctamente	registro de un nuevo material educativo
204 no content	operacion exitosa sin contenido de respuesta	eliminación o retiro exitoso
400 bad request	solicitud incorrecta	datos incompletos o con formato invalido
401 unauthorized	no existe autenticación válida	usuario no autenticado
403 forbidden	el usuario no tiene autorización	el estudiante intenta ejecutar una operación exclusiva del administrador
404 not found	recurso o encontrado	identificador inexistente
500 internal server error	error interno del servidor	falla inesperada durante la ejecución del servicio

14.8. Representación de uno de los servicios mediante SOAP/WSDL

Aunque EduSmart se plantea principalmente mediante servicios REST, una de sus operaciones también podría implementarse utilizando el protocolo **SOAP (Simple Object Access Protocol)** y describirse mediante **WSDL (Web Services Description Language)**.

Para este ejemplo se seleccionó el servicio de recomendaciones académicas, debido a que representa una de las funcionalidades centrales de la plataforma.

14.8.1. Propósito del servicio SOAP

El servicio tendría como finalidad recibir la identificación de una necesidad académica y devolver los recursos educativos relacionados con ella.

Por ejemplo, cuando un estudiante selecciona la necesidad "Organización del tiempo", el servicio podría consultar las relaciones disponibles y devolver materiales que contribuyan a fortalecer esa área.

14.8.2. Operación propuesta

Nombre de la operación: `ObtenerRecomendaciones`.

Datos de entrada:

- Identificador del estudiante.
- Identificador de la necesidad académica.

Datos de salida:

- Identificador de la necesidad.
- Nombre de la necesidad.
- Lista de recursos recomendados.
- Descripción de cada recurso.
- Justificación de la recomendación.

14.8.5. Función del WSDL

El WSDL sería el documento XML que describiría formalmente el servicio SOAP. Allí se especificarán:

- El nombre del servicio.
- Las operaciones disponibles.
- Los mensajes de entrada y salida.
- Los tipos de datos utilizados.
- El protocolo de comunicación.
- La dirección donde estaría publicado el servicio.

De manera conceptual, el WSDL permitiría que otro sistema conociera cómo invocar la operación `Obtener Recomendaciones`, qué información debe enviar y qué estructura tendrá la respuesta.

14.8.6. Diferencias entre REST y SOAP en EduSmart

Característica	REST	SOAP
enfoque	arquitectura orienta a recursos	protocolo de intercambio de mensajes
formato utilizado	principalmente JSON	XML
comunicación	métodos HTTP como GET, POST, PATCH y DETELE	generalmente operaciones mediante mensajes SOAP
descripción del servicio	se documenta	se describe mediante WSDL
complejidad	generalmente más sencillo de implementar en aplicaciones web	requiere una estructura formal de mensajes y contratos
uso propuesto	servicios principales de edusamrt	alternativa para interoperar con sistemas que requiere SOAP

14.9. Relación de los servicios web con la ontología OWL y el grafo Neo4j

Los servicios web propuestos no solamente deben permitir consultar y administrar información, sino también facilitar el acceso a las relaciones semánticas que representan el dominio de EduSmart.

La ontología OWL permitiría definir conceptos como:

- Estudiante.
- Necesidad académica.
- Recurso educativo.
- Categoría.
- Tema.
- Servicio académico.
- Recomendación.
- Asesor académico.

También permitiría establecer relaciones entre estos conceptos, como:

- Un estudiante presenta una necesidad académica.
- Un recurso educativo atiende una necesidad.
- Un recurso pertenece a una categoría.
- Un recurso aborda uno o varios temas.
- Un asesor orienta a un estudiante.

La conexión con los servicios se realizaría de la siguiente manera:

1. El usuario selecciona una necesidad académica desde la interfaz.
2. La interfaz envía una solicitud a la API REST.
3. El servicio de recomendaciones recibe y valida los datos.
4. El componente de consulta semántica consulta las relaciones definidas en la ontología y representadas en el grafo.
5. Neo4j devuelve los recursos y temas relacionados.
6. El servicio organiza los resultados en una respuesta JSON.
7. La interfaz presenta las recomendaciones al estudiante.

14.10. Seguridad y consideraciones técnicas

Para garantizar un funcionamiento adecuado de los servicios web, se proponen las siguientes medidas:

Autenticación: implementar un mecanismo que permita identificar a los usuarios que ingresan a la plataforma.

Autorización: establecer permisos según el perfil. Por ejemplo, los estudiantes podrán consultar recursos, mientras que los administradores tendrán permisos para registrar, modificar o retirar materiales.

Validación de datos: verificar que la información enviada cumpla con los formatos y campos requeridos.

Protección de la comunicación: utilizar HTTPS para proteger la información transmitida entre la interfaz y la API.

Control de errores: implementar respuestas claras para solicitudes incorrectas, recursos inexistentes o fallas internas.

Documentación: mantener una descripción actualizada de los endpoints, parámetros, respuestas y códigos de estado.

Integridad de la información: verificar que las relaciones entre estudiantes, necesidades, temas y recursos sean coherentes con el modelo conceptual y la ontología.

14.11. Conclusión del diseño de servicios web

El diseño de los servicios web de EduSmart permite establecer una estructura organizada para soportar las principales funcionalidades de la plataforma. Mediante el enfoque REST se propone atender la consulta de recursos educativos, la administración de categorías, la identificación de necesidades académicas, la generación de recomendaciones y la exploración de relaciones semánticas.

La definición de los datos de entrada y salida, los métodos HTTP y los perfiles que utilizarían cada servicio facilita comprender cómo se comunicarán los diferentes componentes del sistema.

Asimismo, la explicación de SOAP/WSDL permite reconocer una alternativa de integración basada en mensajes XML y contratos formales de servicios, que podría utilizarse si EduSmart necesitará comunicarse con sistemas institucionales que trabajen bajo este estándar.

En conclusión, los servicios web constituyen un componente fundamental de la arquitectura propuesta, porque permiten conectar la interfaz de usuario con la lógica de negocio, la información académica y los componentes semánticos. Su implementación futura contribuiría a construir una plataforma más organizada, mantenible, interoperable y preparada para ampliar sus funcionalidades.